

Note Detection based on Song Attributes

Κουτσούκης Ιωάννης



Introduction	3
Scope	3
Methodology	4
Data Acquisition	4
Data Prep	4
Model Development & Tuning	6
Machine Learning Models & Comparisons	6
Support Vector Machines (SVM)	6
Random Forests	6
XG Boost	7
LSTM	7
Hyperparameter Tuning: Why Optuna?	8
Comparison	9
Metrics Used	9
Side-to-Side Model Performance	9
Visual Comparison	10
Line Graphs	10
Support Vector Machine	10
Confusion Matrices	12
Support Vector Machine	12
Key Findings & Results	14
Future Work & Next Steps	14
Sources	15

Introduction

In the context of our Machine Learning coursework, I explored the idea of developing an algorithm capable of identifying played musical notes based on their audio characteristics. The primary objective of this project is to lay the groundwork for future applications in music technology, such as pitch correction (autotune), pattern recognition, and musical analysis.

Given the increasing role of machine learning in audio processing, this project aims to analyze the features of audio signals and leverage various ML models to classify musical notes accurately. By comparing different models—such as Support Vector Machines (SVM), Random Forests, XGBoost, and Long Short-Term Memory (LSTM) networks—this study evaluates their effectiveness in recognizing distinct musical notes and assesses their potential for real-world applications.

The findings from this project can serve as a starting point for more advanced implementations in areas like music transcription, intelligent tuning systems, and automated music generation, making it an exciting area of exploration in the field of AI and sound engineering.

Scope

This project focuses on the identification of musical notes from audio recordings using machine learning techniques. The study aims to evaluate various models in terms of accuracy, efficiency, and suitability for real-world applications. The core objectives include:

- **Feature Extraction:** Identifying relevant characteristics of audio signals (e.g., frequency, harmonics, and timbre) that can be used for note classification.
- **Model Comparison:** Evaluating and comparing the performance of SVM, Random Forest, XGBoost, and LSTM in note recognition.
- **Data Processing & Preparation:** Handling raw audio data, performing necessary transformations, and ensuring model readiness.

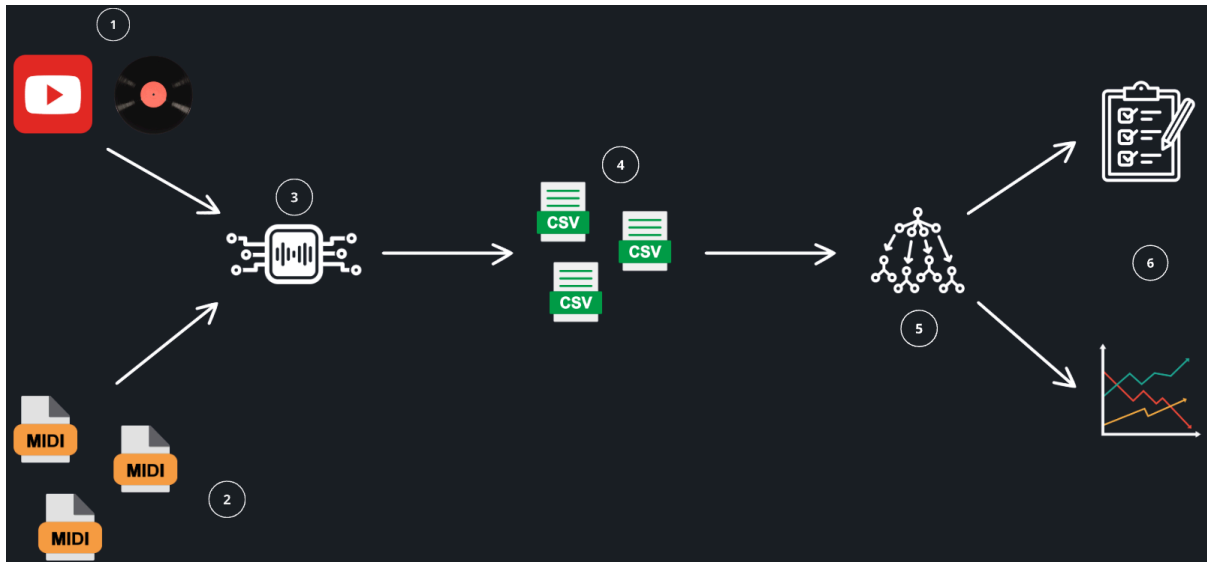
Limitations & Exclusions:

- The study does not focus on chord recognition or polyphonic music (multiple notes played simultaneously).
- Only a predefined set of musical notes will be classified, and no real-time implementation is considered in this phase.
- The project does not involve deep learning-based speech recognition or natural language processing.

The results of this research will provide insights into the feasibility of using machine learning for musical note identification and serve as a foundation for further developments, such as autotuning, pitch correction, and intelligent music pattern detection.

Methodology

The methodology for this study follows a structured pipeline covering data acquisition, feature extraction, model training, hyperparameter tuning, and evaluation.



Data Acquisition

Audio Sources: Karaoke and instrumental versions of songs were sourced from YouTube.

Ground Truth Labels: Corresponding MIDI files were obtained from online sequencers, which provided the actual played notes.

Feature Extraction:

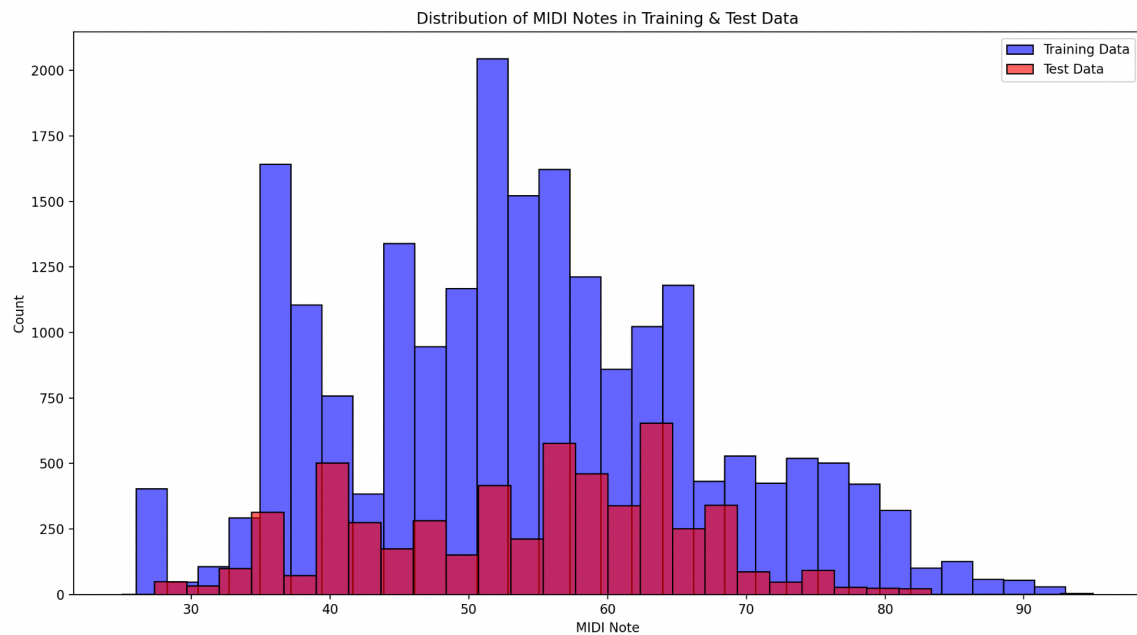
- Audio processing was performed using the Librosa library, extracting key features such as:
 - Pitch & Power
 - Spectral Centroid
 - Timeframes & Temporal Evolution
- The MIDI data was processed using Mido, extracting the corresponding notes as ground truth labels.

Data Prep

The extracted features and MIDI notes were synchronized and stored as CSV files, ensuring that each song had a structured dataset.

To prevent inconsistencies, missing values, and underrepresented notes, oversampling techniques like SMOTE were applied.

Clustering was used to reduce the number of distinct notes and outliers were removed to enhance data quality.



Model Development & Tuning

Machine Learning Models & Comparisons

Four machine learning models were selected, each with a unique approach to note classification. Their selection was based on interpretability, adaptability to structured audio data, and computational feasibility.

Support Vector Machines (SVM)

Why SVM?

- Well-suited for small-to-medium datasets and high-dimensional feature spaces.
- Effective for structured classification tasks like MIDI note identification.

Implementation Steps:

1. Checks Class Distribution in Training & Test Data
2. Balances Classes Using SMOTE or Random Oversampling
3. Standardizes Features (critical for SVM performance)
4. Tunes Hyperparameters Using Optuna or Grid Search (optional)
5. Trains the Final SVM Model
6. Makes Predictions & Generates a Classification Report

Random Forests

Why Random Forest?

- Works well with structured categorical data like MIDI notes.
- Less sensitive to outliers and feature scaling issues compared to other models.

Implementation Steps:

1. Encodes MIDI Notes as Numeric Labels
2. Handles Class Imbalance Using SMOTE or Random Oversampling
3. Standardizes Features (important for Random Forest)
4. Builds a Default Random Forest Model
5. Tunes Hyperparameters Using Optuna or Grid Search (optional)
6. Trains the Final Model
7. Makes Predictions & Generates a Classification Report

XG Boost

Why XGBoost?

- Handles imbalanced datasets well and performs effectively even with missing values.
- Highly efficient with tree-based feature selection and learning mechanisms.

Implementation Steps:

1. Encodes MIDI Notes as Numeric Labels
2. Handles Unseen Labels in the Test Set
3. Balances Classes Using SMOTE or Oversampling
4. Standardizes Features (important for XGBoost)
5. Builds a Default XGBoost Model
6. Tunes Hyperparameters Using Optuna (optional)
7. Trains the Final Model
8. Calibrates the Model for Better Probability Predictions (optional)
9. Makes Predictions & Generates a Classification Report

LSTM

Why LSTM?

- Best suited for sequential audio data, capturing temporal dependencies in note transitions.
- Can learn the evolution of musical patterns over time.

Implementation Steps:

1. Prepares the Data
2. Standardizes features (critical for LSTMs).
3. Reshapes the data for LSTM compatibility.
4. Builds an LSTM Model
5. Defines an LSTM network with:
 - a. Multiple LSTM layers 
 - b. Dropout layers to prevent overfitting 
 - c. Dense layers to output MIDI note predictions 
 - d. Uses Adam / RMSprop / SGD optimizers for tuning.
6. Hyperparameter Tuning with Optuna
7. Trains the Final Model with the best hyperparameters.
8. Makes Predictions using the trained model.

Hyperparameter Tuning: Why Optuna?

Several hyperparameter tuning approaches were considered:

Grid Search:

- Exhaustive but slow—tested every combination but led to long runtimes.

Randomized Grid Search:

- Improved efficiency by sampling parameter sets, but still required manual effort.

Optuna Optimization Framework:

- Adaptive tuning—learns from past trials to suggest better configurations.
- Reduced computational cost compared to Grid Search.
- Supports early stopping, preventing unnecessary evaluations.
- Chosen for its efficiency, automation, and ability to find optimal configurations faster.

Each model was trained with multiple hyperparameter configurations, evaluated, and fine-tuned using Optuna's optimization process.

Example SVM Optuna-Tuning

```
Performing Optuna Optimization for best parameters...
[I 2025-02-11 00:32:25,219] A new study created in memory with name: no-name-c3811661-22ba-43e1-a676-0c4d64639d12
[I 2025-02-11 00:43:23,979] Trial 0 finished with value: 0.23729121278140886 and parameters: {'C': 1.3597482723170737, 'gamma': 'auto', 'kernel': 'rbf'}.
Best is trial 0 with value: 0.23729121278140886.
[I 2025-02-11 00:54:21,937] Trial 1 finished with value: 0.2276861361828682 and parameters: {'C': 0.9879371077376831, 'gamma': 'scale', 'kernel': 'rbf'}.
Best is trial 0 with value: 0.23729121278140886.
[I 2025-02-11 01:05:24,335] Trial 2 finished with value: 0.214804444029463636 and parameters: {'C': 0.6399140082476769, 'gamma': 'auto', 'kernel': 'rbf'}.
Best is trial 0 with value: 0.23729121278140886.
[I 2025-02-11 01:16:09,500] Trial 3 finished with value: 0.2789881384652626 and parameters: {'C': 3.7309318510043545, 'gamma': 'scale', 'kernel': 'rbf'}.
Best is trial 3 with value: 0.2789881384652626.
[I 2025-02-11 01:27:06,098] Trial 4 finished with value: 0.2367551959055227 and parameters: {'C': 1.2621884553157592, 'gamma': 'auto', 'kernel': 'rbf'}.
Best is trial 3 with value: 0.2789881384652626.
Best parameters found: {'C': 3.7309318510043545, 'gamma': 'scale', 'kernel': 'rbf'}

All Optuna Tested Parameter Combinations:
  C gamma kernel accuracy
0 1.359748 auto rbf 0.237291
1 0.987937 scale rbf 0.227686
2 0.639914 auto rbf 0.214804
3 3.730932 scale rbf 0.278988
4 1.262188 auto rbf 0.236755
```

Example Random Forests Optuna-Tuning

```
Performing Optuna Optimization for Random Forest...
[I 2025-02-11 11:19:39,194] A new study created in memory with name: no-name-85dc253c-fbd2-44cf-b01d-315f7ba286b3
[I 2025-02-11 11:19:42,301] Trial 0 finished with value: 0.6500155617802676 and parameters: {'n_estimators': 143, 'max_depth': 15, 'min_samples_split': 16, 'min_samples_leaf': 6}. Best is trial 0 with value: 0.6500155617802676.
[I 2025-02-11 11:19:45,451] Trial 1 finished with value: 0.7268475291351109 and parameters: {'n_estimators': 119, 'max_depth': 20, 'min_samples_split': 4, 'min_samples_leaf': 8}. Best is trial 1 with value: 0.7268475291351109.
[I 2025-02-11 11:19:46,892] Trial 2 finished with value: 0.7060811979112633 and parameters: {'n_estimators': 51, 'max_depth': 21, 'min_samples_split': 13, 'min_samples_leaf': 10}. Best is trial 1 with value: 0.7268475291351109.
[I 2025-02-11 11:19:50,057] Trial 3 finished with value: 0.699043815056887 and parameters: {'n_estimators': 138, 'max_depth': 17, 'min_samples_split': 18, 'min_samples_leaf': 4}. Best is trial 1 with value: 0.7268475291351109.
[I 2025-02-11 11:19:53,267] Trial 4 finished with value: 0.6500242072137496 and parameters: {'n_estimators': 143, 'max_depth': 16, 'min_samples_split': 13, 'min_samples_leaf': 10}. Best is trial 1 with value: 0.7268475291351109.
Best parameters found: {'n_estimators': 119, 'max_depth': 20, 'min_samples_split': 4, 'min_samples_leaf': 8}
```

Example XG Boost Optuna-Tuning

```
Performing Optuna Optimization for XGBoost...
[I 2025-02-11 12:49:25,661] A new study created in memory with name: no-name-490ee66f-bafa-44db-8964-0b544c2b090d
[I 2025-02-11 12:50:09,609] Trial 0 finished with value: 0.33578863644223117 and parameters: {'n_estimators': 170, 'learning_rate': 0.0186868486460109, 'max_depth': 3, 'gamma': 3.7485157577809263, 'subsample': 0.5187119156304377, 'colsample_bytree': 0.550888672243387, 'reg_alpha': 0.9067067599554828, 'reg_lambda': 0.6947573985657322}. Best is trial 0 with value: 0.33578863644223117.
[I 2025-02-11 12:51:12,172] Trial 1 finished with value: 0.650119306982052 and parameters: {'n_estimators': 170, 'learning_rate': 0.04022860122017199, 'max_depth': 9, 'gamma': 3.150384221667608, 'subsample': 0.7351804583597767, 'colsample_bytree': 0.6128643596059977, 'reg_alpha': 0.5708199147284433, 'reg_lambda': 0.8284479158429177}. Best is trial 1 with value: 0.650119306982052.
[I 2025-02-11 12:51:33,273] Trial 2 finished with value: 0.34283466473008956 and parameters: {'n_estimators': 82, 'learning_rate': 0.040515864307957054, 'max_depth': 3, 'gamma': 3.080521303298198, 'subsample': 0.6422861893852827, 'colsample_bytree': 0.7249267294298103, 'reg_alpha': 0.14154342692887067, 'reg_lambda': 0.19259686060473502}. Best is trial 1 with value: 0.650119306982052.
[I 2025-02-11 12:52:02,792] Trial 3 finished with value: 0.6095203513504167 and parameters: {'n_estimators': 91, 'learning_rate': 0.12548999439248418, 'max_depth': 6, 'gamma': 3.4997404271723327, 'subsample': 0.6727832821489625, 'colsample_bytree': 0.9680289671486064, 'reg_alpha': 0.24370206168708552, 'reg_lambda': 0.9907024604058233}. Best is trial 1 with value: 0.650119306982052.
[I 2025-02-11 12:52:57,956] Trial 4 finished with value: 0.7596569491994328 and parameters: {'n_estimators': 192, 'learning_rate': 0.18291923603616525, 'max_depth': 8, 'gamma': 1.0749663602324695, 'subsample': 0.5017577250946178, 'colsample_bytree': 0.8971716114423031, 'reg_alpha': 0.9712864399327187, 'reg_lambda': 0.22853237065633625}. Best is trial 4 with value: 0.7596569491994328.
Best parameters found: {'n_estimators': 192, 'learning_rate': 0.18291923603616525, 'max_depth': 8, 'gamma': 1.0749663602324695, 'subsample': 0.5017577250946178, 'colsample_bytree': 0.8971716114423031, 'reg_alpha': 0.9712864399327187, 'reg_lambda': 0.22853237065633625}
```

Comparison

Metrics Used

Metric	Explanation
Accuracy 🎯	Percentage of correct note predictions.
Precision 🎵	Measures how many predicted notes were actually correct.
Recall ✅	Measures how many actual notes were correctly predicted.
F1 Score ⚖️	Balances between precision and recall.
AUC-ROC 📊	Evaluates the model's ability to distinguish between different MIDI notes.
Mean Absolute Error (MAE) 🎵	Measures how far the predicted note is from the actual note. Lower values = better pitch accuracy.
Top-3 Accuracy 🎵	Checks if the correct note is among the top 3 predictions , useful for musical applications.
Training & Prediction Time ⌚⚡	Measures efficiency of model execution.
Model Size 💾	Determines feasibility for deployment (smaller models preferred for real-time applications).

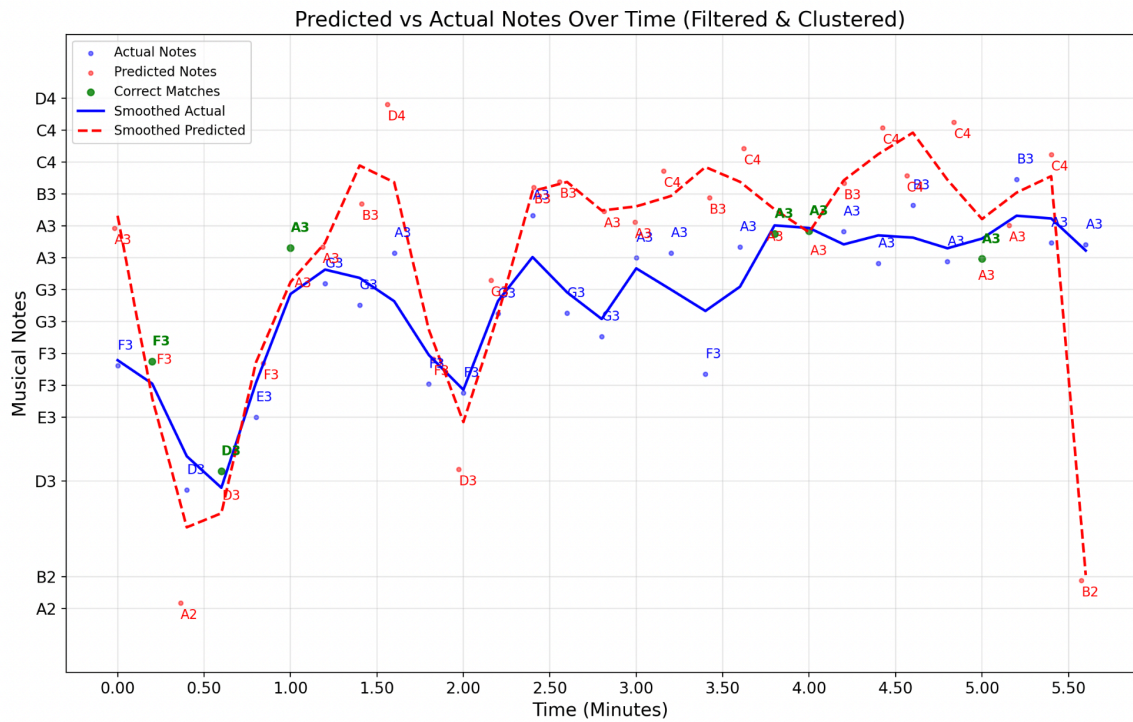
Side-to-Side Model Performance

Model	✅ Accuracy	🎯 Precision	↺ Recall	📊 F1 Score	🎵 MAE (Note Distance)	🏆 Top-3 Accuracy	⌚ Training Time (sec)	⚡ Prediction Time (sec)
♦ SVM	0.0353	4	0.0353	0.0072	11.2518	N/A	55.6013	10.5257
🚀 XGBoost	0.0685	0.0743	0.0685	0.0656	10.9973	0.1532	7.9776	61
🧠 LSTM	0.0312	0.0017	0.0312	3	⌚ 10.6018	N/A	⌚ 0.0	0.1532
🌲 Random Forest	✅ 0.0767	✅ 0.1087	✅ 0.0767	✅ 0.074	11.0335	✅ 0.1691	0.6721	⌚ 0.0363

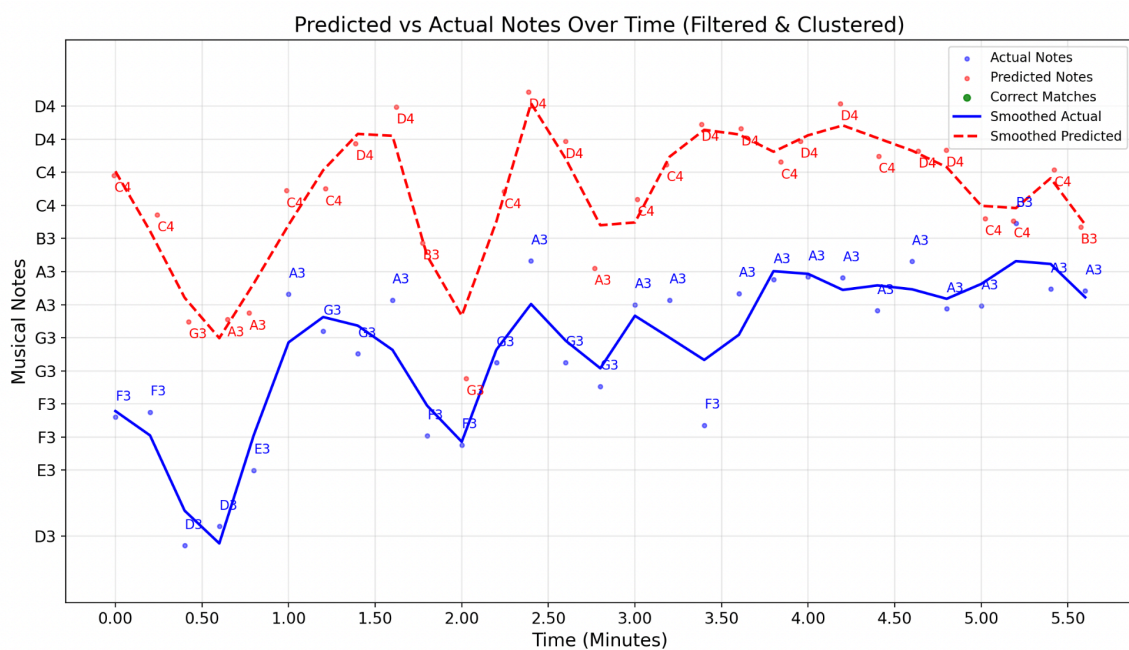
Visual Comparison

Line Graphs

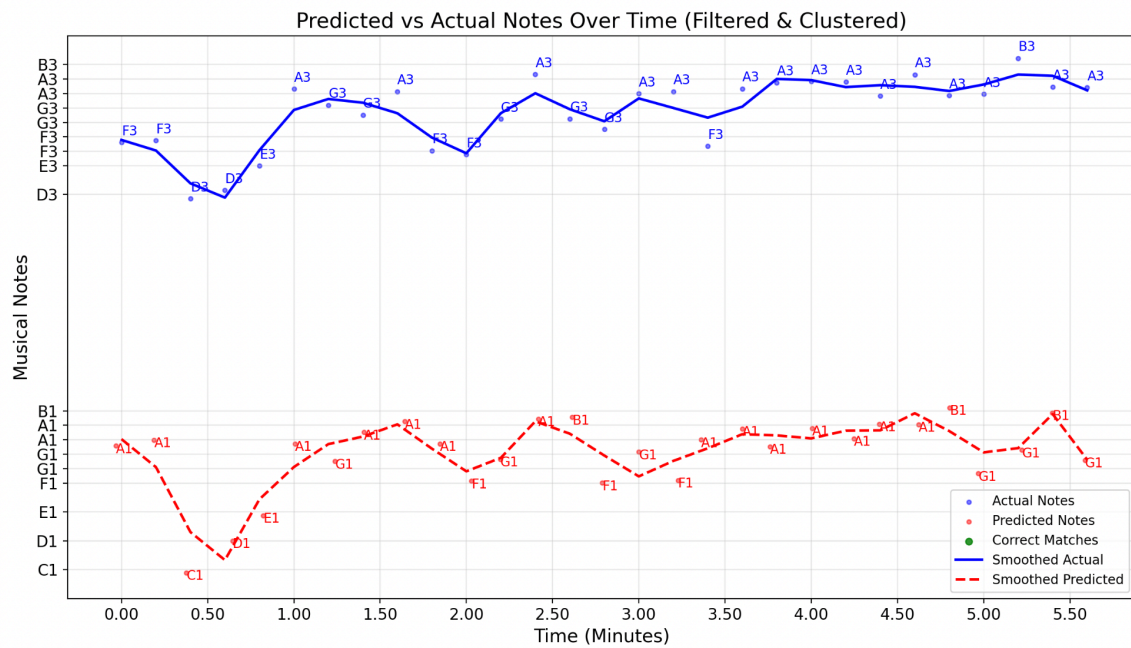
Support Vector Machine



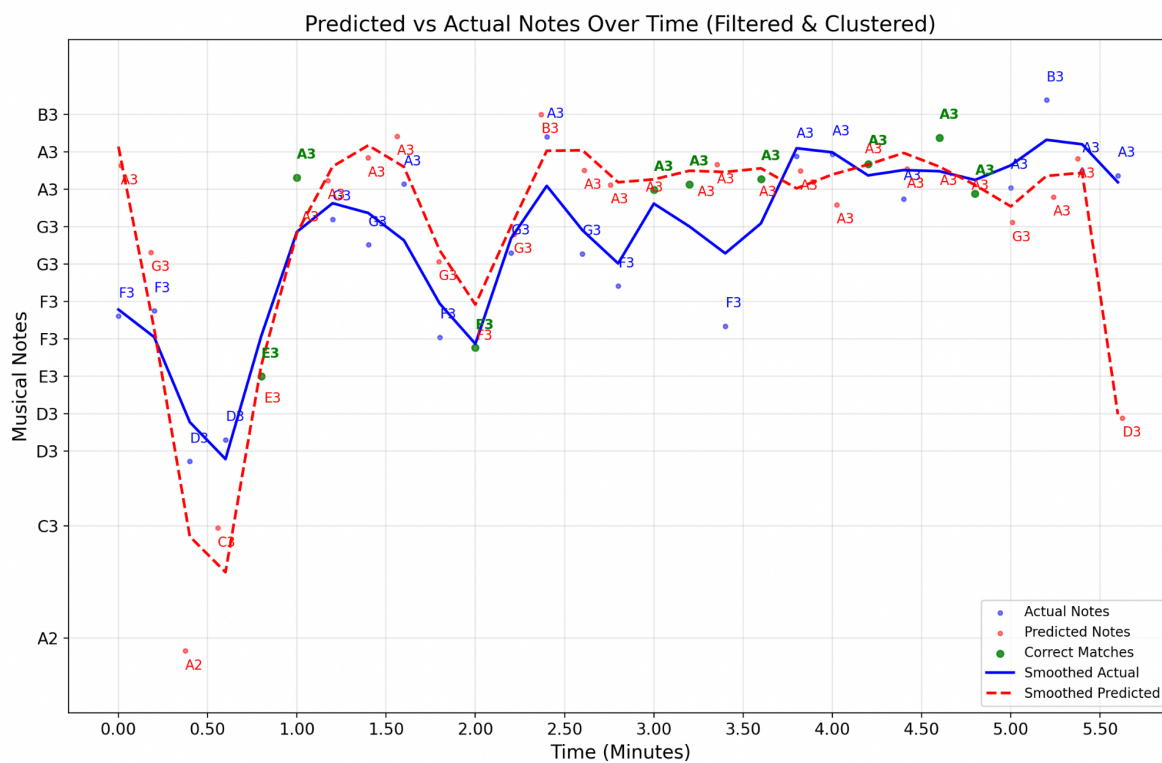
Random forests



XG Boost

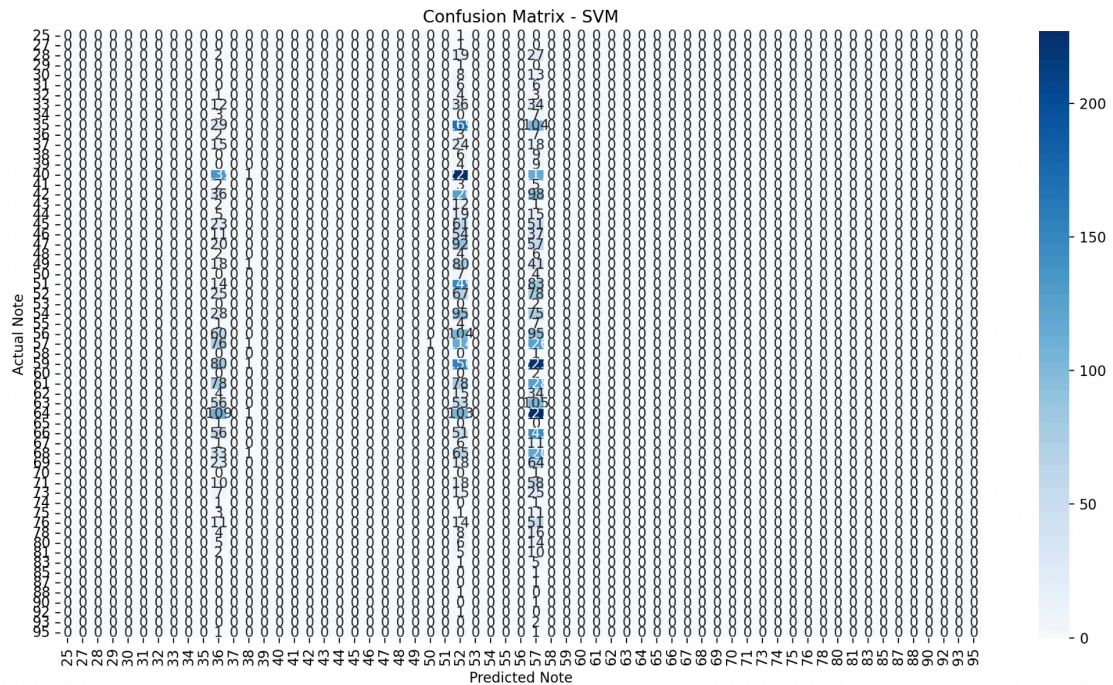


LSTM

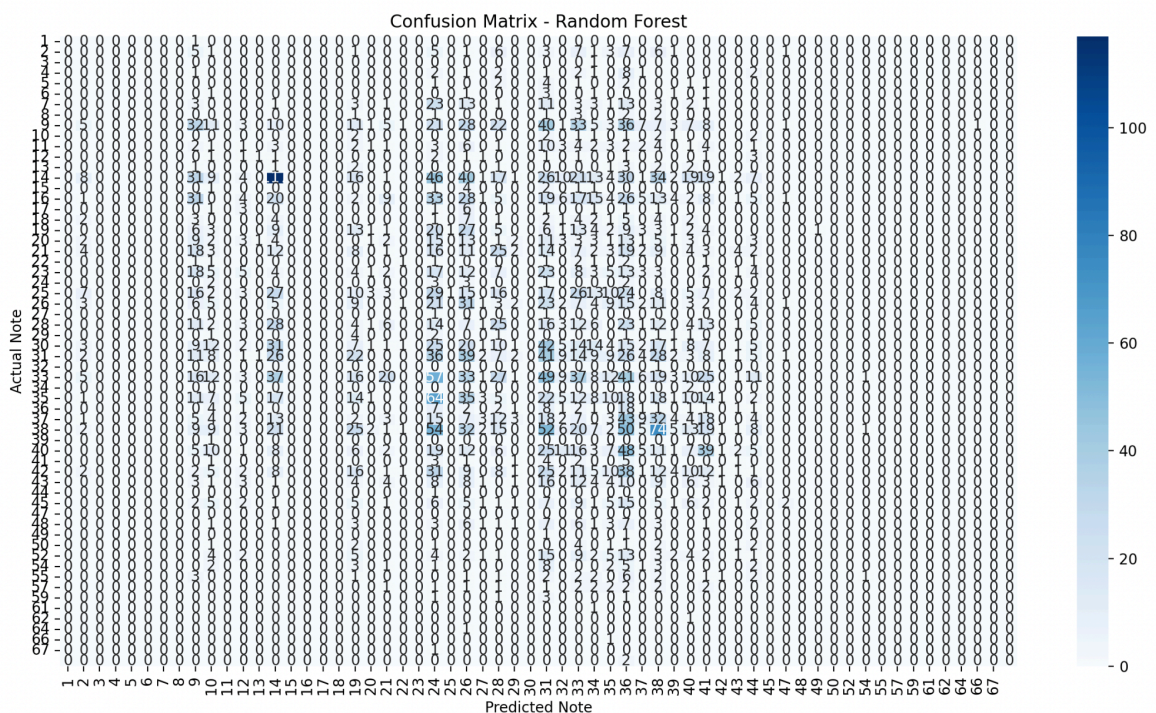


Confusion Matrices

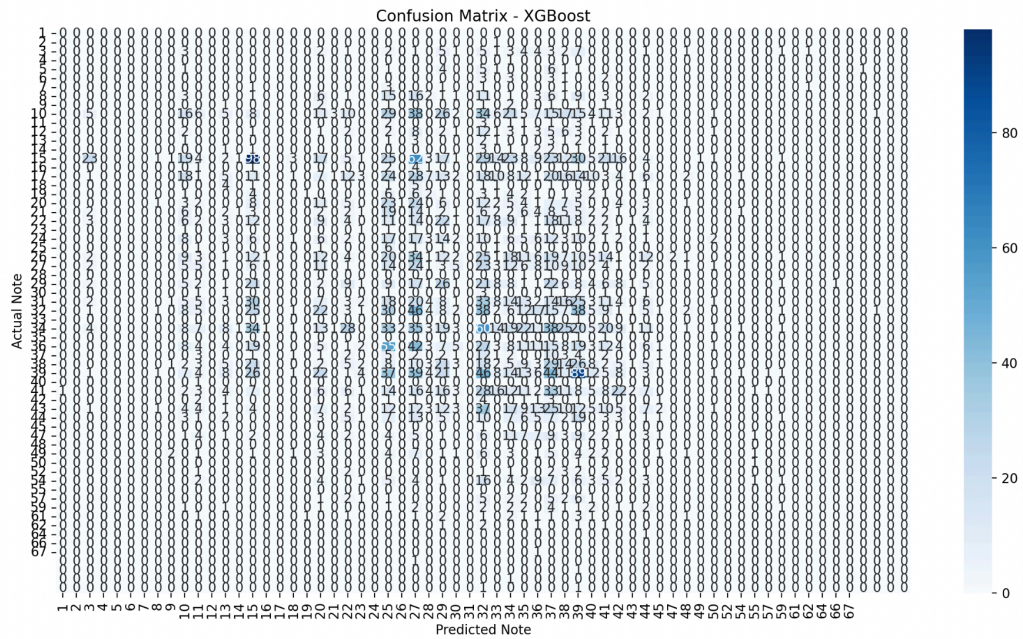
Support Vector Machine



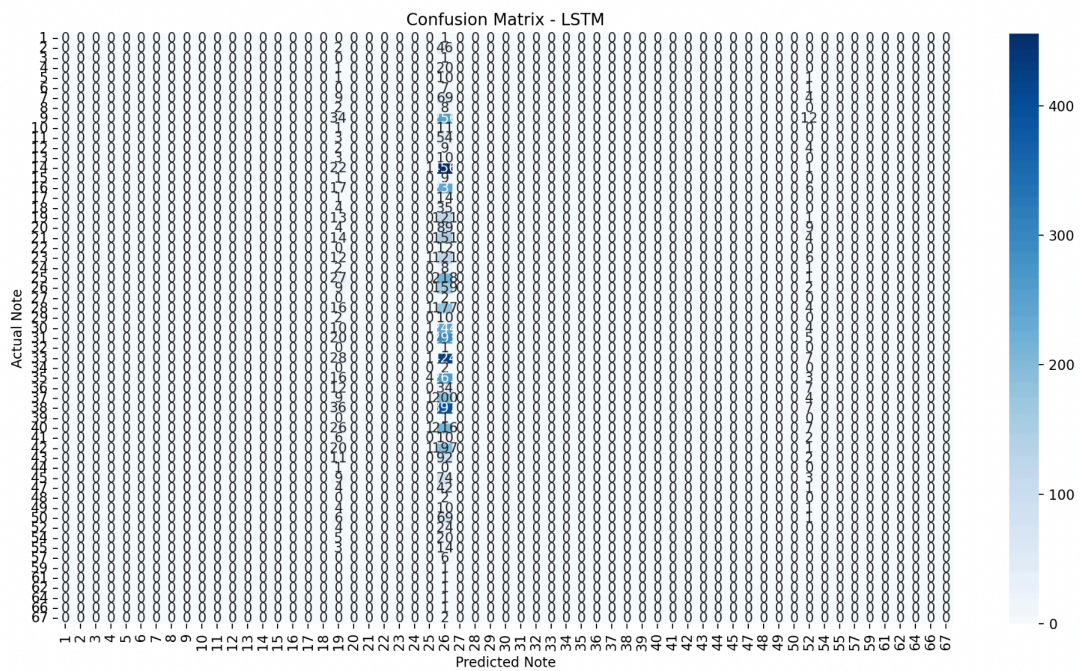
Random forests



XG Boost



LSTM



Key Findings & Results

1. No single model outperformed all others—each had its strengths:
 - a. LSTM models captured sequential dependencies but struggled with highly varied datasets.
 - b. XGBoost & Random Forest performed well in structured classification but had trouble distinguishing similar notes.
 - c. SVM was reliable in well-balanced datasets but sensitive to noise.
2. Data preprocessing played a major role—clustering and outlier removal improved all models.
3. Even with tuning, challenges remained—pitch variation and dataset complexity impacted model generalization.
4. Visualization of results (as seen in smoothed graphs) provided deeper insights than raw metrics alone.

Future Work & Next Steps

1. To improve performance and expand practical applications, the following areas can be explored:
 - a. Training Set Enhancement – Expanding the dataset with more varied samples, including human-recorded instrumentals.
 - b. More Advanced Tuning – Exploring neural architecture search (NAS) or Bayesian optimization for deep learning models.
 - c. Individual Tests per Genre – Investigating how different genres (e.g., classical vs. pop) impact model performance.
 - d. Instrumental vs. Vocal Tests – Evaluating whether vocals-only datasets lead to different results compared to instrumentals.
2. Web-App Deployment – Developing a web interface where users can upload their own songs for note recognition.
3. Real-Time Note Identification – Exploring low-latency models for live instrument tracking.

Sources

Librosa (Audio Feature Extraction Library)

- Official Documentation
- **Used for:** Feature extraction (pitch, spectral centroid, MFCCs, etc.)

Mido (MIDI File Processing Library)

- [Official Documentation](#)
- **Used for:** Reading MIDI files to obtain ground truth labels

Scikit-learn (Machine Learning Models & SMOTE Handling)

- [Official Documentation](#)
- **Used for:** SVM, Random Forest, data preprocessing

XGBoost (Optimized Gradient Boosting Algorithm)

- [Official Documentation](#)
- **Used for:** Efficient tree-based classification

TensorFlow/Keras (Deep Learning for LSTM Model)

- [Official Documentation](#)
- **Used for:** Training LSTM models for sequential note recognition

Optuna (Hyperparameter Optimization Framework)

- [Official Documentation](#)
- **Used for:** Tuning model hyperparameters efficiently